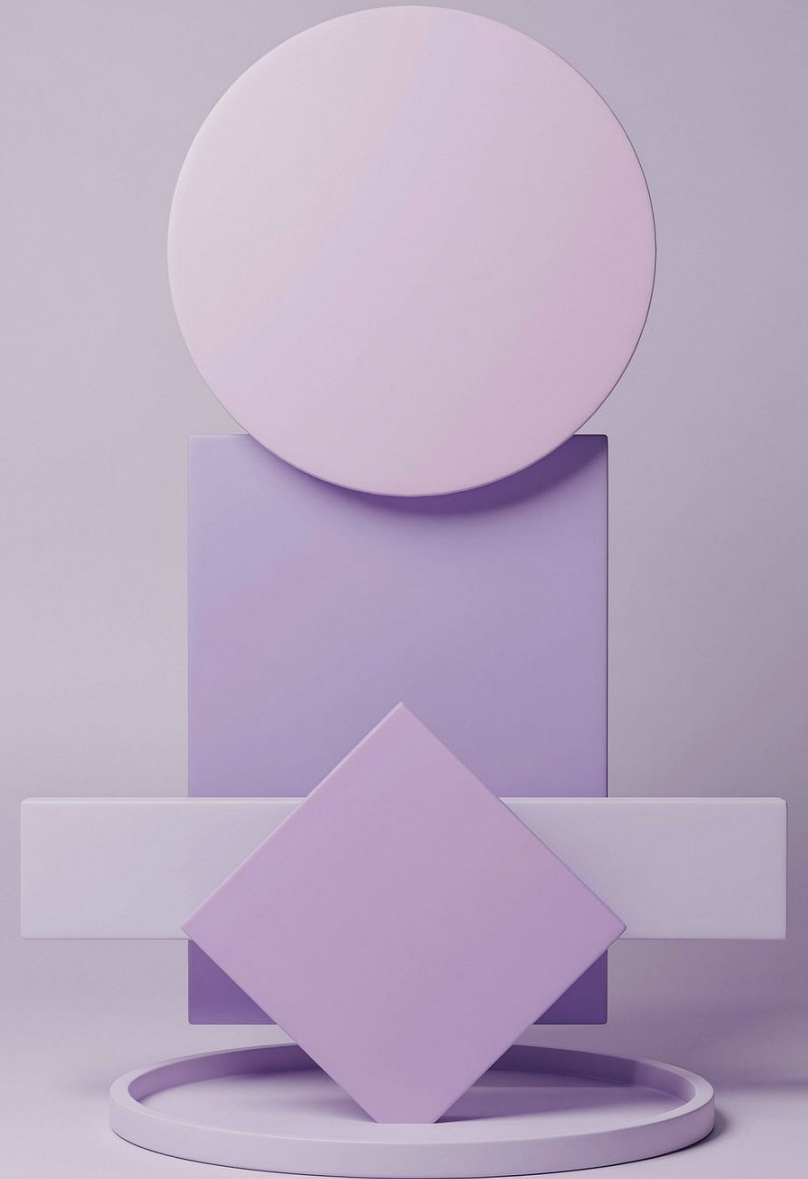


CIERRE DE SERIE

Polimorfismo + Clase Asociación Ternaria

El cierre de la serie

Dos conceptos clave que elevan el modelo orientado a objetos al siguiente nivel.



CONCEPTO CENTRAL

¿Qué es el polimorfismo?

Un **método abstracto** vive en la superclase sin ninguna implementación propia. Es solo una promesa: "todas mis subclases van a poder responder a este llamado". Cada subclase se encarga de implementarlo a su manera.

La clave está en el código que **llama** al método: nunca pregunta de qué subclase se trata. Simplemente invoca el método, y cada objeto responde según su propia versión. No hay ifs, no hay preguntas de tipo en el medio.

Esa es la diferencia entre **tener herencia** y **usar polimorfismo de verdad**: la herencia es la estructura; el polimorfismo es que el mismo llamado produzca resultados distintos según el objeto real.

La fórmula del polimorfismo

- Superclase define el método abstracto
- Cada subclase lo implementa a su manera
- El llamado es idéntico para todos
- El resultado varía según el objeto real
- Sin ifs ni preguntas de tipo

¿Qué es una clase asociación ternaria?

Ya vimos que una relación puede depender de **dos clases a la vez** (clase asociación binaria): el dato no pertenece a ninguna de las dos clases por separado, sino a la combinación de ambas. Una clase asociación ternaria es el **mismo patrón**, un escalón más complejo.

01

Clase asociación binaria

El dato depende de la combinación de **dos** clases al mismo tiempo. Solo tiene sentido cuando se combinan ambas — no pertenece a ninguna sola.

02

Clase asociación ternaria

El dato depende de la combinación de **tres** clases al mismo tiempo. El mismo test aplica: ¿este dato depende de la combinación, y no de una sola clase?

03

El test sigue siendo el mismo

"¿Este dato depende de la combinación, y no de una sola clase?" — pero ahora la combinación tiene **tres participantes** en lugar de dos.

La complejidad aumenta, pero el razonamiento es el mismo. Si el dato solo tiene sentido cuando se juntan los tres, necesita una clase asociación ternaria.

EL ENUNCIADO

Cosmética Natural: el caso de negocio

"Una empresa de cosmética natural vende productos a través de representantes. Cada venta queda registrada con el representante, el producto y la fecha, y opcionalmente se completa después con el cliente final. Los representantes pueden ser vendedores comunes o líderes de equipo. En las reuniones periódicas se calcula la comisión de cada representante: la de un vendedor común depende de sus propias ventas; la de un líder depende de sus propias ventas y, además, de las ventas de todo su equipo."

Entidades principales

Representante, Producto, Cliente

Jerarquía de representantes

Vendedor común vs. Líder de equipo

La clase asociación

El Ticket: venta con fecha, representante y producto

El cálculo clave

Comisión periódica, con reglas distintas por tipo

El Ticket ternario y el cliente diferido

Cada venta liga a la vez tres clases: **Representante**, **Producto** y **Cliente**. El dato (la venta) no pertenece a ninguna de las tres por separado — solo tiene sentido cuando se combinan las tres.

Pero hay un matiz importante: el **Ticket nace sin cliente**. La venta se registra primero con representante y producto, y el cliente se agrega después, si es que la venta se completa. Por eso el cliente tiene multiplicidad **0..1** en esta relación ternaria.

Y hay una **regla de integridad** crítica: el cliente que se agrega después tiene que pertenecer a la cartera de ese mismo representante. No se puede completar una venta con el cliente de otro representante.

Multiplicidades del Ticket

- **Representante:** obligatorio desde el inicio (1)
- **Producto:** obligatorio desde el inicio (1)
- **Cliente:** diferido, se agrega después (0..1)

Regla de integridad

- El cliente debe pertenecer a la cartera del representante de esa venta
- No se puede cruzar con clientes de otros representantes

Representante: herencia {disjoint, complete} + método abstracto

Representante es una superclase abstracta con dos subclases fijas. La restricción **{disjoint, complete}** dice exactamente esto: todo representante es uno o el otro, nunca ambos, y no existe una tercera opción.

Superclase: Representante

Abstracta — no se puede instanciar directamente. Define el método abstracto **calcular_comision()** sin ninguna implementación propia. Es solo la promesa de que todas las subclases van a saber calcular su comisión.

Subclase: Vendedor

Implementa **calcular_comision()** a su manera: basándose únicamente en sus propias ventas. Es una de las dos opciones posibles, excluyente con Líder.

Subclase: Líder

Implementa **calcular_comision()** a su manera: considerando sus propias ventas y las de su equipo. Es la otra opción posible, excluyente con Vendedor.

- ❏ **{disjoint, complete}** — disjoint significa que las subclases no se solapan (no podés ser Vendedor y Líder al mismo tiempo); complete significa que no hay representantes que no sean ni uno ni otro.

La palabra que delata el override: *"además"*

Cuando el enunciado dice que un Líder calcula su comisión **"de la misma manera... y además"**, esa palabra es la señal más clara de que hay un override real. No es una operación completamente diferente — es la misma operación, pero haciendo más trabajo.

Vendedor: `calcular_comision()`

Calcula la comisión a partir de sus **propias ventas**, nada más.
Define la base del comportamiento.

- Suma sus ventas del período
- Aplica el porcentaje correspondiente
- Resultado: comisión individual

Líder: `calcular_comision()` (*override*)

Calcula la misma base que el Vendedor... **y además** suma un porcentaje de las ventas de todo su equipo.

- Suma sus propias ventas del período
- Suma las ventas de todo su equipo
- Aplica los porcentajes correspondientes

Ambas subclases responden al mismo llamado — **`calcular_comision()`** — pero producen resultados distintos según cuál sea el objeto real. Eso es polimorfismo.

¿POR QUÉ IMPORTA?

Despachar sin preguntar el tipo

Imaginá una lista mixta con vendedores y líderes juntos, y tenés que calcular la comisión total de todos. Aquí es donde el polimorfismo muestra su valor real.

✗ Sin polimorfismo

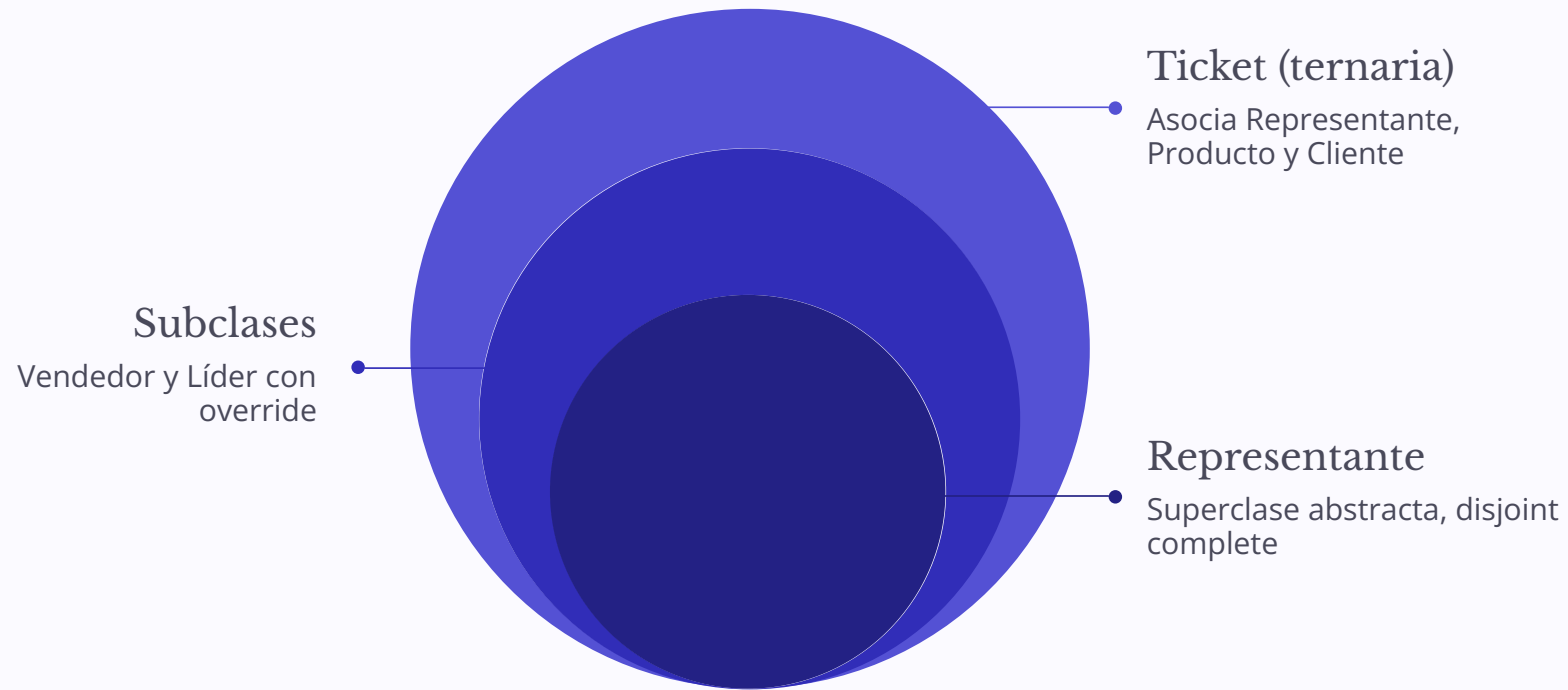
Recorrés la lista preguntando "¿sos Vendedor o sos Líder?" con un **if en cada vuelta**. Aplicás una fórmula distinta a mano según el tipo. El código que suma las comisiones necesita conocer la lógica interna de cada subclase. Si agregás un tercer tipo de representante, tenés que modificar ese código.

✓ Con polimorfismo

El código simplemente le pide a **cada representante que calcule SU comisión**. Cada uno responde con la fórmula que le corresponde. El código que acumula el total no sabe nada de las subclases — ni necesita saberlo. Si agregás un tercer tipo, no tocás ese código.

"El polimorfismo no es solo elegancia: es la diferencia entre un sistema que crece limpiamente y uno que se vuelve frágil cada vez que hay que extenderlo."

El modelo completo: diagrama de cierre



Representante {abstracta}

Superclase con **calcular_comision()** abstracto. Herencia {disjoint, complete}: Vendedor y Líder, nunca ambos, siempre uno.

Ticket (ternaria)

Clase asociación que depende de Representante (1), Producto (1) y Cliente (**0..1**, diferido). El cliente se agrega después si la venta se completa.

Override por subclase

Vendedor: comisión por ventas propias. Líder: comisión por ventas propias + **equipo**. Mismo llamado, resultados distintos.